

Parquet Query Agent - Architecture & Business Guide

Executive Summary

What is this?

An AI agent that lets business users ask questions in plain English and automatically generates SQL queries to answer them from parquet data files.

Key Benefits

- ✓ **No SQL knowledge required** - Users ask questions naturally ("Show me top customers")
- ✓ **Self-service analytics** - No waiting for data team to write queries
- ✓ **Safe & governed** - Built-in guardrails prevent destructive operations
- ✓ **Business context aware** - Uses glossaries to understand your terminology

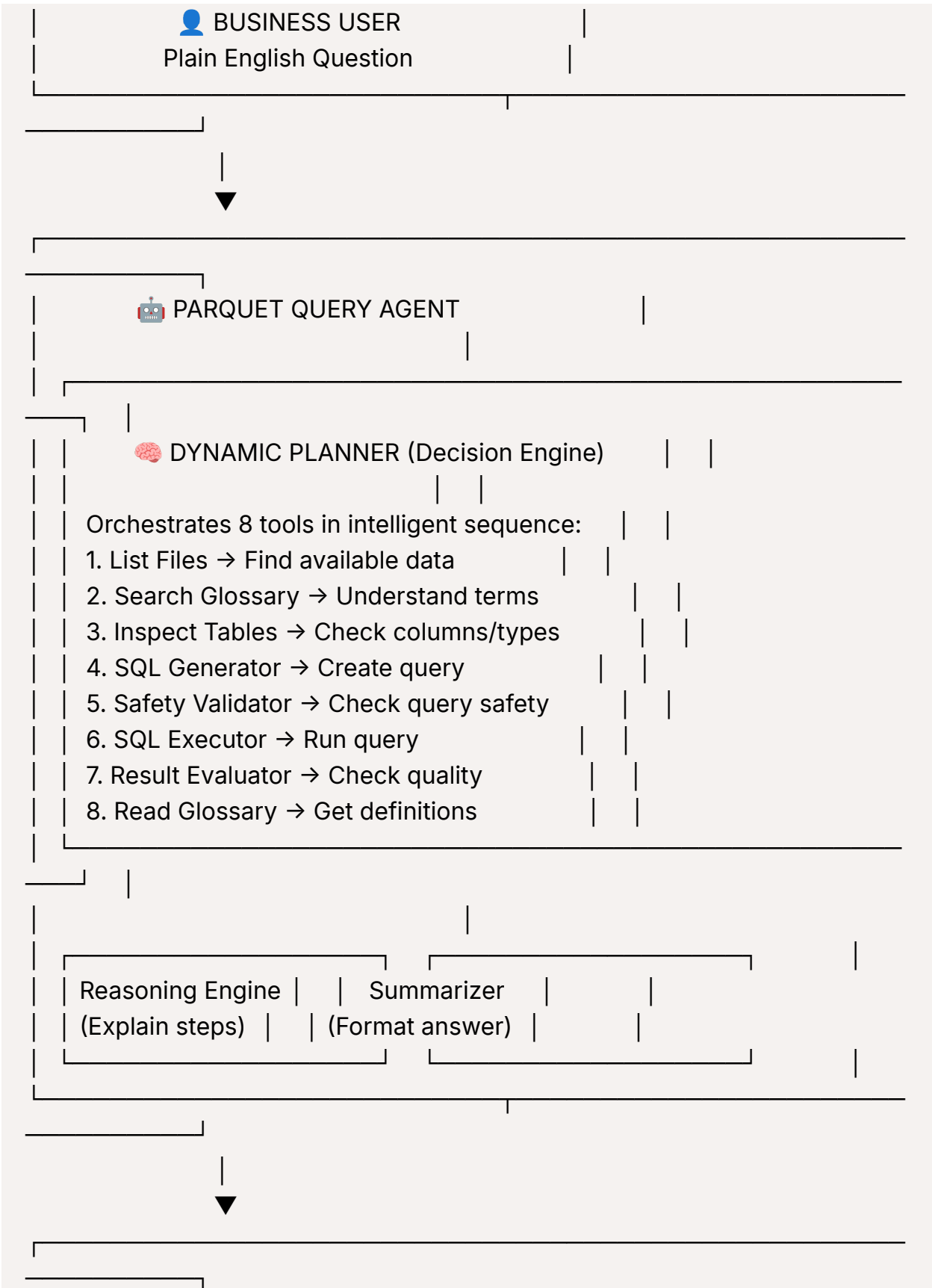
How it works

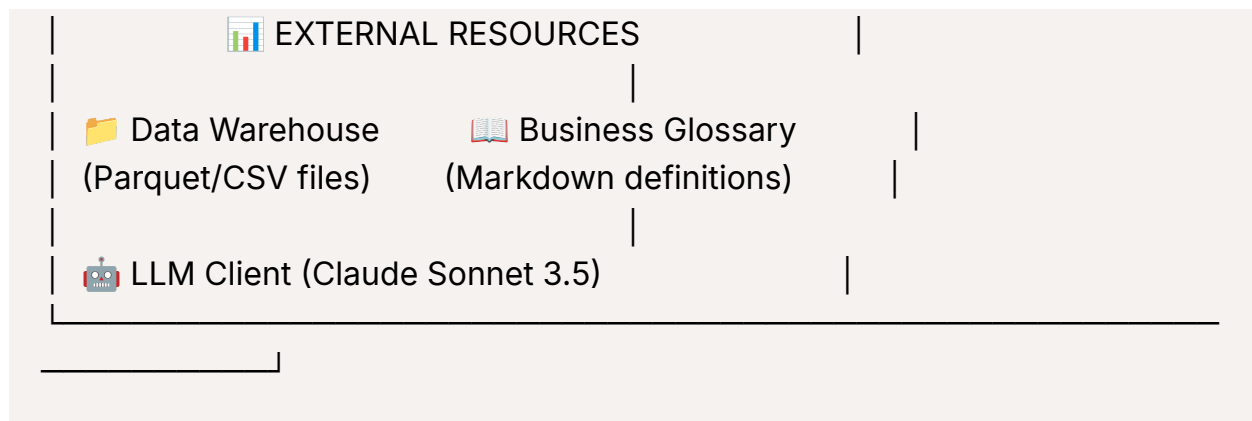
- 1 User asks a question in plain English
- 2 Agent reads business glossaries to understand your data
- 3 Agent generates and validates SQL query
- 4 Agent executes query and formats results
- 5 User gets answer in seconds

High-Level Architecture

System Components







Orchestrator Component Architecture

Input Layer

User Query → Plain English question

Previous State → Optional (for follow-up questions)

Orchestrator Layer (agent_v2.py)

Follow-up Classifier → Determines if query is:

- **NEW_QUERY** - Fresh question
- **DRILL_DOWN** - Digging deeper into previous results
- **REFINE** - Modifying previous query

State Initializer → Creates **PlannerState** object to track everything

Dynamic Planner (dynamic_planner.py)

Planning Loop → Max 50 iterations with 10 intelligence layers:

1. **Clarity Check** → Clear / Blocked / Defaultable
2. **Discovery** → Search + List + Inspect
3. **Candidate Selection** → Rank top 5 tables
4. **Mapping Synthesis** → Business → SQL guidance
5. **SQL Generation** → Pure SQL generator

6. **Multi-layer Validation** → 5 safety checks
7. **SQL Execution** → DuckDB on parquet
8. **Result Evaluation** → Quality check
9. **Repair Mechanism** → Fix issues (max 3 attempts)
10. **Strict Stop Criteria** → Ensure completion

Output Layer

Reasoning Engine → Generates thinking trace

Summarizer → Formats answer + commentary

Response Envelope → {answer, thinking, trace, meta}

External Resources

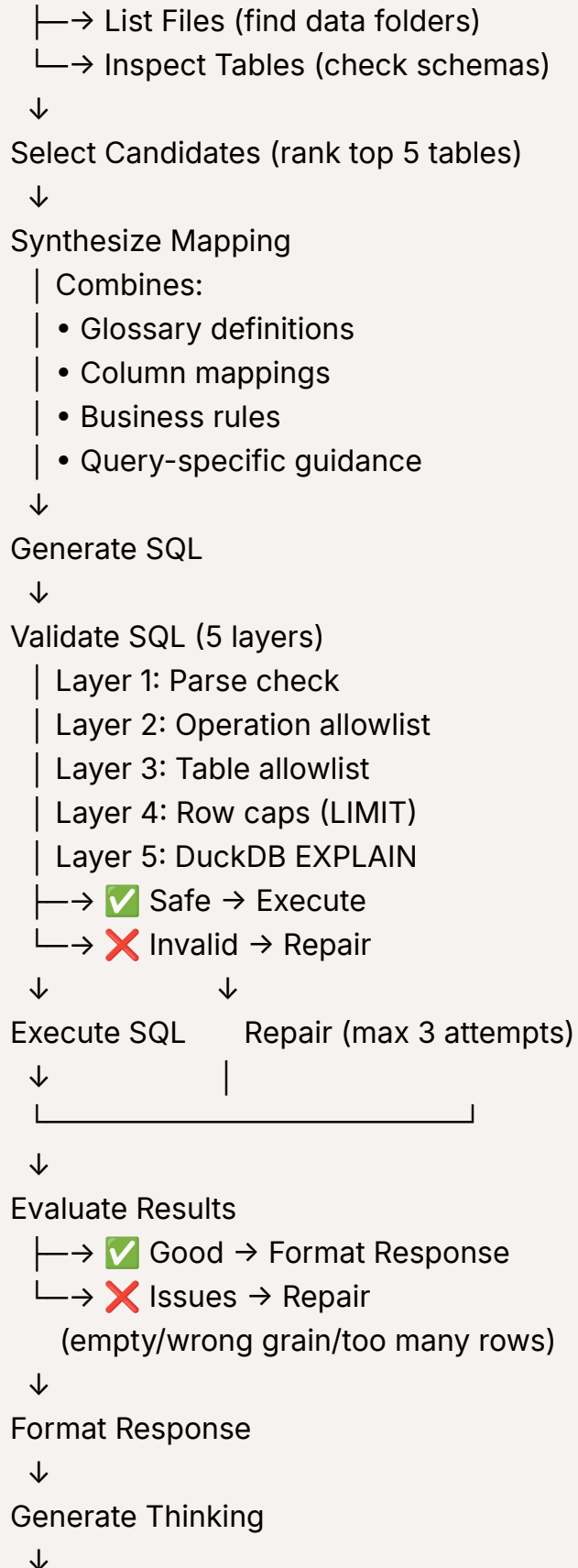
- **Tools Registry** → 8 discovery/execution tools
- **Data Warehouse** → Parquet/CSV files
- **Business Glossaries** → Markdown definitions
- **LLM Client** → Claude Sonnet 3.5

Dynamic Planner Workflow (State Machine)

Flow Diagram

```
graph TD
    START --> UserQuestion[User Question]
    UserQuestion --> InitState[Initialize State]
    InitState --> ClarityCheck[Clarity Check]
    ClarityCheck --> Clear[Clear]
    Clear --> Continue[Continue]
    ClarityCheck --> Ambiguous[Ambiguous]
    Ambiguous --> AskUser[Ask User]
    AskUser --> WAIT[WAIT]
    WAIT --> DiscoveryPhase[Discovery Phase]
    DiscoveryPhase --> SearchGlossary[Search Glossary find business terms]
```

START → User Question
↓
Initialize State
↓
Clarity Check
├→  Clear → Continue
└→  Ambiguous → Ask User → WAIT
↓
Discovery Phase
├→ Search Glossary (find business terms)



Generate Summary



END → Return to User

Orchestrator Decision Flow (Business PM View)

Decision Tree

User Question



Is question clear?

├→ NO → Ask user for clarification → WAIT

└→ YES



Search business glossary



Found relevant terms?

├→ NO → List all data folders

└→ YES → List all data folders



Inspect top 5 candidate tables



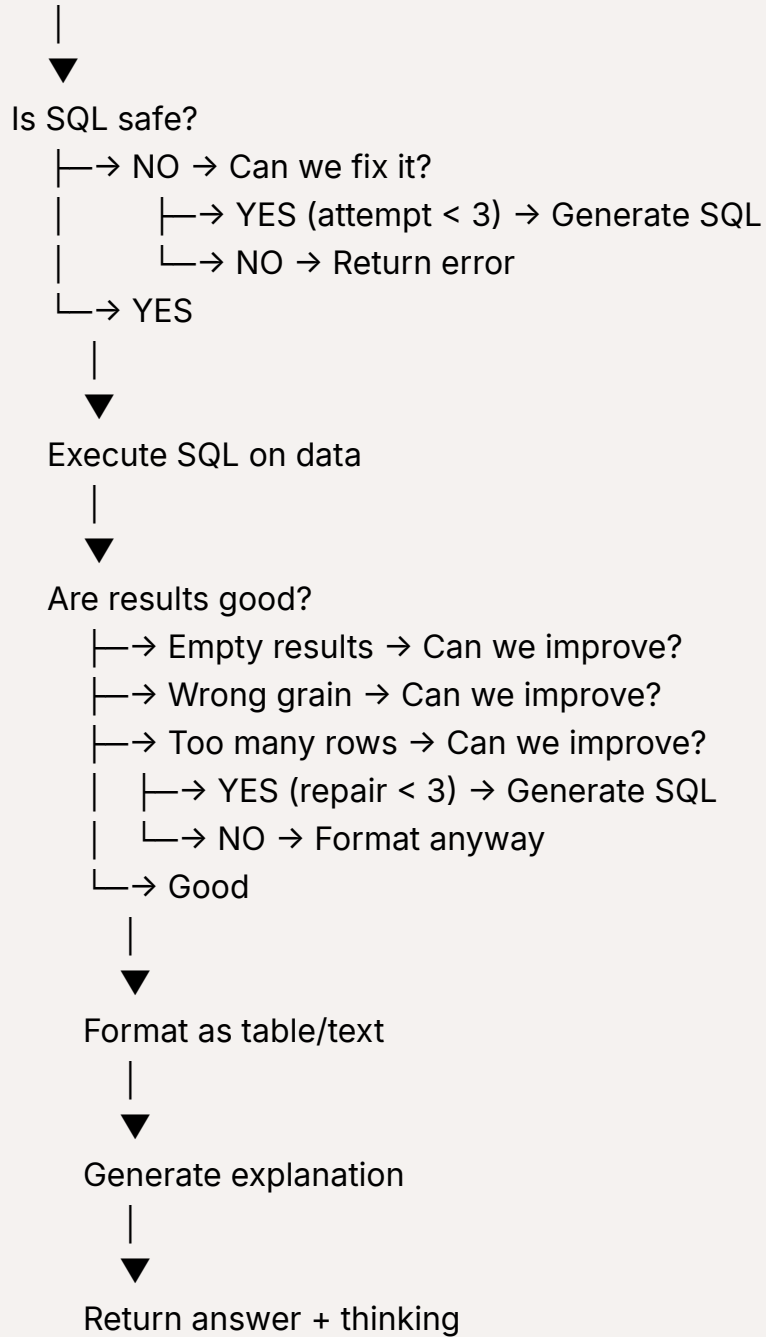
Rank tables by relevance score



Build SQL mapping guidance



Generate SQL query



Key Orchestrator Intelligence Layers

1. Clarity Precheck

What it does: Checks if the question is answerable

Decisions:

-  **Clear** - Has all needed info, proceed
-  **Defaultable** - Missing info but can assume defaults
-  **Blocked** - Must ask user for clarification

Example:

- "Show me exposure" →  Blocked (which domain? which date?)
 - "Show me CCR exposure for today" →  Clear
-

2. Candidate Table Selection

What it does: Finds the 5 most relevant tables

Scoring based on:

- Glossary match score (business term found in glossary)
- Column name overlap (query terms match column names)
- Domain relevance (CCR query → CCR tables)

Output: Ranked list with scores (e.g., customer_exposure.csv: 0.95)

3. Mapping Synthesis

What it does: Translates business terms → SQL instructions

Combines:

- **Metrics:** "exposure" → `net_exposure` column
- **Dimensions:** "by customer" → `GROUP BY customer_id`
- **Time filters:** "today" → `ccr/2025-12-01/`
- **Business rules:** "Use net_exposure, NOT breach_amount"
- **Query guidance:** Relevant columns, similar examples, avoid patterns

Output: Structured mapping context for SQL generator

4. Multi-Layer Validation

What it does: Ensures SQL is safe before execution

5 Layers:

1. **Parse** - Is it valid SQL syntax?
2. **Operations** - Only SELECT allowed (no DELETE/UPDATE/DROP)
3. **Table allowlist** - Only reads from candidate tables
4. **Row caps** - Has LIMIT clause (prevents massive results)
5. **DuckDB EXPLAIN** - No cross-joins or expensive operations

Result: Pass/Fail + specific error messages

5. Result Evaluation

What it does: Checks if SQL results make sense

Checks:

-  **Empty results** - Query too restrictive?
-  **Wrong grain** - Missing GROUP BY?
-  **Too many rows** - Need better filters?
-  **Good** - Results match expectations

Triggers repair if issues found

6. Repair Mechanism

What it does: Fixes issues automatically (max 3 attempts)

Strategies:

- **Syntax error** → Fix SQL syntax
- **Missing column** → Use correct column from glossary
- **Empty results** → Relax filters or change date
- **Wrong grain** → Add GROUP BY

Bounded: Stops after 3 repair attempts to prevent infinite loops

7. Strict Stop Criteria

What it does: Ensures agent only finishes when job is done

Agent can FINISH only if:

-  SQL was generated
-  SQL was validated (passed all 5 layers)
-  SQL was executed
-  Results were evaluated as "good"

Prevents premature exits

How The Agent Works (Step-by-Step)

Phase 1: Discovery

User asks: "Show me top customers by revenue"

Agent does:

1. **List Files** - Scans data warehouse folders (ecommerce/, ccr/, etc.)
 2. **Search Glossary** - Looks up "customers" and "revenue" in business glossaries
 3. **Inspect Tables** - Checks columns in relevant files (customers.parquet, orders.parquet)
-

Phase 2: Planning

Agent thinks:

- "Revenue = sum of order amounts"
 - "Need to join customers + orders tables"
 - "Group by customer, order by revenue descending"
-

Phase 3: Generation & Validation

Agent does:

1. **Generate SQL** - Creates query using discovered tables/columns

2. **Validate Safety** - Checks:

- No DELETE/UPDATE/DROP commands
 - Only reads allowed tables
 - Has row limits (prevents massive results)
 - No expensive operations (cross-joins)
-

Phase 4: Execution & Response

Agent does:

1. **Execute SQL** - Runs query on parquet files
 2. **Evaluate Results** - Checks if results make sense
 3. **Format Answer** - Presents data as table
 4. **Explain Thinking** - Shows what it did and why
-

Guardrails & Safety

Loop Prevention

- **No-repeat guards** - Can't call same tool with same args twice
- **SQL history** - Can't execute same SQL twice
- **Progress validation** - Must make progress every 3 iterations
- **Attempt caps** - Max 5 SQL generation attempts, max 3 repairs

Safety Validation

- **Read-only** - Only SELECT queries allowed
- **Table allowlist** - Can only read from discovered candidate tables
- **Row limits** - All queries must have LIMIT clause
- **Explain analysis** - DuckDB checks for expensive operations

Error Handling

- **Bounded retries** - Won't loop forever
 - **Graceful degradation** - Returns partial results if repair fails
 - **Clear error messages** - Tells user what went wrong
-

Business Glossary Guide

Why Glossaries Matter

Without glossaries, the agent guesses what your terms mean. With glossaries, it **knows** your business rules.

Example:

✗ Without glossary: "exposure" could mean gross_exposure, net_exposure, or breach_amount

✓ With glossary: "exposure means net_exposure after netting, NOT breach_amount"



What Business Users Need to Provide

For Each Data Folder/Table

Create a markdown file in: `business_glossary/{domain}/tables/{table_name}.md`

Example: `business_glossary/ccr/tables/customer_exposure.md`



Template: Business Glossary File

Section 1: What This Table Contains

[2-3 sentence description of what this data represents]

Example:

"Customer exposure data shows our credit risk to each counterparty. Updated daily. Includes both gross and net exposure after netting."

Section 2: File Location

Path: `ccr/2025-12-01/customer_exposure.csv`

Partitioning: Date-partitioned (one folder per date)

Update Frequency: Daily at 6am

Section 3: Key Columns

customer_id

- **What it is:** Unique identifier for counterparty
- **Use for:** Identifying which customer, grouping by customer
- **Example values:** CUST001, CUST002

net_exposure

- **What it is:** Credit exposure after netting (USE THIS FOR EXPOSURE QUERIES)
- **Use for:** Calculating total exposure, finding high-risk customers
- **Business rule:** This is the official exposure metric, not gross_exposure

gross_exposure

- **What it is:** Total exposure before netting
- **Use for:** Understanding exposure before risk mitigation
- **When to use:** Only when specifically asked for "gross" or "pre-netting"

limit_breach_flag

- **What it is:** Y/N flag indicating if customer exceeded limit
 - **Use for:** Finding customers over limit
 - **Don't use for:** Exposure calculations (use net_exposure instead)
-

Section 4: Common Queries

"Show me customer exposure"

```
SELECT customer_id, net_exposure
FROM ccr/2025-12-01/customer_exposure.csv
ORDER BY net_exposure DESC
```

"Which customers breached limits?"

```
SELECT customer_id, net_exposure, breach_amount
FROM ccr/2025-12-01/customer_exposure.csv
WHERE limit_breach_flag = 'Y'
```

Section 5: Important Rules

1. **Always use net_exposure for exposure queries** (not gross_exposure or breach_amount)
2. **Date partitioning:** Use specific date folder (ccr/YYYY-MM-DD/)
3. **Breaches vs Exposure:** breach_amount is only for limit breaches, not general exposure

Section 6: Related Tables

- `customer_limits.csv` - Credit limits per customer
- `trade_data.csv` - Individual trades contributing to exposure



What to Include in Each Section

1. What This Table Contains

- **Purpose:** High-level business description
- **Include:** What the data represents, update frequency, business context
- **Keep it simple:** 2-3 sentences a business user would understand

2. File Location

- **Path:** Exact relative path to the data file
- **Partitioning:** Is it date-partitioned? Time-partitioned? Not partitioned?
- **Update schedule:** When is this data refreshed?

3. Key Columns

For each important column, provide:

- **What it is:** Plain English definition
- **Use for:** When to use this column (SELECT, WHERE, GROUP BY, JOIN)
- **Business rules:** Special handling (e.g., "use net not gross")
- **Example values:** Sample data to show format

Which columns to document:

-  Metrics (amounts, counts, rates)
-  Dimensions (customer_id, product_id, desk)
-  Flags (breach_flag, active_flag)
-  Dates (as_of_date, trade_date)
-  Skip technical columns (internal IDs, audit timestamps)

4. Common Queries

Provide 2-5 example queries:

- Show actual SQL the agent should generate
- Cover typical business questions
- Include correct column usage
- Show proper date/partition handling

5. Important Rules

Document business logic:

- Which column to use when (e.g., "use net_exposure not gross_exposure")
- Calculation rules (e.g., "revenue = price * quantity")

- Filtering rules (e.g., "exclude cancelled orders")
- Join rules (e.g., "join on customer_id")

6. Related Tables

List connected data:

- Tables that can be joined
- Tables with related metrics
- Tables needed for complete analysis

Quality Checklist

Before submitting a glossary file, verify:

- ☐ File path is correct and uses relative paths (not absolute)
- ☐ Partitioning is clearly explained (date-partitioned vs not)
- ☐ All metric columns have "use for" guidance
- ☐ Business rules are explicit (no ambiguity)
- ☐ Example queries use correct columns
- ☐ Common mistakes are documented in "avoid" section
- ☐ Related tables are listed for joins

Benefits of Good Glossaries

Without Glossary	With Glossary
Agent guesses column meanings	Agent knows exact business definitions
Uses wrong columns (breach_amount for exposure)	Uses correct columns (net_exposure)
Misses date partitioning	Handles partitions correctly
Generic error messages	Specific business context
40% accuracy	95% accuracy

Getting Started

1. **Identify your data domains** (e.g., CCR, ecommerce, market risk)
2. **List key tables** in each domain
3. **Assign owners** - Who knows this data best?
4. **Use template above** to document each table
5. **Test with agent** - Ask questions, refine glossaries based on results

 **Time estimate:** 30-60 minutes per table (one-time effort)

Success Metrics

Agent Performance

- **Accuracy:** % of queries that return correct results
- **Speed:** Average time from question to answer
- **Safety:** % of queries that pass all validation layers
- **Self-service rate:** % of queries answered without human intervention

Business Impact

- **Time saved:** Hours per week not waiting for data team
 - **Query volume:** Number of questions asked per user
 - **User satisfaction:** NPS or feedback scores
 - **Coverage:** % of business domains with complete glossaries
-

Support & Troubleshooting

Common Issues

"Agent used wrong column"

→ Check glossary has explicit "use for" guidance and business rules

"Agent can't find my data"

→ Verify file path in glossary matches actual data location

"Query returns empty results"

→ Check date partitioning is correctly documented in glossary

"Agent asks for clarification too often"

→ Add common query examples to glossary to teach agent patterns

Getting Help

1. Check agent's "thinking" output to see what it tried
2. Review glossary for the tables it used
3. Compare agent's SQL to expected SQL
4. Update glossary with corrections
5. Test again

Appendix: Technical Details

Data Warehouse Structure

```
data_warehouse/
├── ecommerce/           # Non-partitioned domain
│   ├── customers.parquet
│   ├── orders.parquet
│   └── products.parquet
├── ccr/                 # Date-partitioned domain
│   ├── 2025-12-01/
│   │   ├── customer_exposure.csv
│   │   └── trade_data.csv
│   ├── 2025-12-02/
│   │   └── ...
│   └── ...
└── mr_exposure/        # Date-partitioned domain
    └── ...
```

Glossary Structure

```
business_glossary/  
├── ccr/  
│   └── tables/  
│       ├── customer_exposure.md  
│       ├── customer_limits.md  
│       └── trade_data.md  
├── ecommerce/  
│   └── tables/  
│       ├── customers.md  
│       ├── orders.md  
│       └── products.md  
└── mr_exposure/  
    └── tables/  
        └── ...
```

Agent Components

- **Dynamic Planner:** Decision-making orchestrator (LLM-driven)
- **Tools:** Pure functions for discovery, validation, execution
- **Reasoning Engine:** Explains what agent did (rule-based)
- **Summarizer:** Formats results for users (rule-based)
- **LLM Client:** Claude Sonnet 3.5 for planning, Haiku for parsing

Version History

- **v3.0** (Current) - Single-brain architecture with dynamic planner
- **v2.0** - Multi-agent DAG architecture (deprecated)
- **v1.0** - Fixed workflow (deprecated)

💡 Pro Tip: Import this document into Notion and use the toggle blocks feature to collapse sections for easier navigation. Add callout blocks for important

| warnings and tips.